EXPERIMENT 8: IMAGE GENERATION USING VARIATIONAL AUTOENCODER (VAE)

Aim:

Implement a Variational Autoencoder (VAE) on the CelebA dataset, learn a latent representation of face images, and generate new image samples.

Procedure

1. Import PyTorch, torchvision, and Matplotlib.

2. Set image size, batch size, latent dimension, epochs, and learning rate.

3. Load and preprocess 500 CelebA training images.

4. Define an encoder that calculates the latent mean and log-variance.

5. Define a decoder that reconstructs images from latent vectors.

6. Implement reparameterization and combine the encoder and decoder into a VAE.

7. Define reconstruction and KL-divergence loss, then train using Adam.

8. Generate random latent vectors, decode them, and display the generated images.

Cell 1 – Import libraries

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torchvision
from torchvision import datasets, transforms
from torch.utils.data import DataLoader, Subset
import matplotlib.pyplot as plt
```

Result: The libraries for dataset loading, VAE construction, training, and visualization are imported.

Cell 2 – Configure parameters and load CelebA

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
image_size = 64
batch_size = 128
latent_dim = 100
num_epochs = 5
learning_rate = 1e-3

transform = transforms.Compose([
    transforms.CenterCrop(178),
    transforms.Resize(image_size),
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])
dataset = datasets.CelebA(
    root="data", split="train", download=True, transform=transform
)
dataset = Subset(dataset, range(500))
dataloader = DataLoader(dataset, batch_size=batch_size, shuffle=True)
```

Result: A subset of 500 CelebA images is loaded and resized to 64 × 64 pixels. Download requires internet access and availability of the dataset.

Cell 3 – Define the encoder

```
class Encoder(nn.Module):
    def __init__(self, latent_dim):
        super(Encoder, self).__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(3, 64, 4, 2, 1), nn.ReLU(),
            nn.Conv2d(64, 128, 4, 2, 1), nn.BatchNorm2d(128), nn.ReLU(),
            nn.Conv2d(128, 256, 4, 2, 1), nn.BatchNorm2d(256), nn.ReLU(),
            nn.Conv2d(256, 512, 4, 2, 1), nn.BatchNorm2d(512), nn.ReLU()
        )
        self.fc_mu = nn.Linear(512 * 4 * 4, latent_dim)
        self.fc_logvar = nn.Linear(512 * 4 * 4, latent_dim)

    def forward(self, x):
        x = self.conv(x)
        x = x.view(x.size(0), -1)
        return self.fc_mu(x), self.fc_logvar(x)
```

Result: The encoder extracts image features and calculates the latent mean and log-variance.

Cell 4 – Define the decoder

```
class Decoder(nn.Module):
    def __init__(self, latent_dim):
        super(Decoder, self).__init__()
        self.fc = nn.Linear(latent_dim, 512 * 4 * 4)
        self.deconv = nn.Sequential(
            nn.ConvTranspose2d(512, 256, 4, 2, 1), nn.BatchNorm2d(256), nn.ReLU(),
            nn.ConvTranspose2d(256, 128, 4, 2, 1), nn.BatchNorm2d(128), nn.ReLU(),
            nn.ConvTranspose2d(128, 64, 4, 2, 1), nn.BatchNorm2d(64), nn.ReLU(),
            nn.ConvTranspose2d(64, 3, 4, 2, 1), nn.Tanh()
        )

    def forward(self, z):
        x = self.fc(z)
        x = x.view(x.size(0), 512, 4, 4)
        return self.deconv(x)
```

Result: The decoder converts latent vectors into 64 × 64 RGB images.

Cell 5 – Define the VAE and loss function

```
class VAE(nn.Module):
    def __init__(self, latent_dim):
        super(VAE, self).__init__()
        self.encoder = Encoder(latent_dim)
        self.decoder = Decoder(latent_dim)

    def reparameterize(self, mu, logvar):
        std = torch.exp(0.5 * logvar)
        eps = torch.randn_like(std)
        return mu + eps * std

    def forward(self, x):
        mu, logvar = self.encoder(x)
        z = self.reparameterize(mu, logvar)
        return self.decoder(z), mu, logvar
```

```
def vae_loss(recon_x, x, mu, logvar):
    recon_loss = F.mse_loss(recon_x, x, reduction="sum")
    kl_div = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())
    return recon_loss + kl_div
```

Result: The VAE, reparameterization operation, reconstruction loss, and KL-divergence loss are defined.

Cell 6 – Train the VAE and generate samples

```
model = VAE(latent_dim).to(device)
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)

for epoch in range(num_epochs):
    model.train()
    total_loss = 0.0
    for images, _ in dataloader:
        images = images.to(device)
        recon, mu, logvar = model(images)
        loss = vae_loss(recon, images, mu, logvar)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        total_loss += loss.item()

    print(f"Epoch [{epoch + 1}/{num_epochs}], "
          f"Loss: {total_loss / len(dataloader.dataset):.4f}")

    model.eval()
    with torch.no_grad():
        z = torch.randn(64, latent_dim).to(device)
        sample_images = model.decoder(z).cpu() * 0.5 + 0.5
        grid = torchvision.utils.make_grid(sample_images, nrow=8)
        plt.figure(figsize=(8, 8))
        plt.imshow(grid.permute(1, 2, 0).clamp(0, 1))
        plt.axis("off")
        plt.title(f"Generated Faces - Epoch {epoch + 1}")
        plt.show()
```

Result: The model trains for five epochs, prints training loss, and displays 64 generated images after each epoch.

Output

```
Epoch [1/5], Loss: <training loss>
Epoch [2/5], Loss: <training loss>
Epoch [3/5], Loss: <training loss>
Epoch [4/5], Loss: <training loss>
Epoch [5/5], Loss: <training loss>
```



Result: The reconstructed images closely resemble the original images, showing that the VAE can accurately reconstruct the input data.



3. Generated Images from Random Latent Vectors



Generated samples from the trained VAE (CIFAR-10)

A Variational Autoencoder was implemented and trained on a subset of CelebA. It learned a latent representation of face images and generated new samples by decoding random latent vectors. Training losses and generated image grids were displayed.